

# Módulo SOAP para clasificar el catálogo de una librería

---

SOAP | WSDL | XML | XSD | Flask | PostgreSQL | WS-Security | SOAP Fault | Interoperabilidad

|                     |  |
|---------------------|--|
| <b>Nombre</b>       |  |
| <b>Matrícula</b>    |  |
| <b>Grupo y hora</b> |  |
| <b>Fecha</b>        |  |

## EJERCICIO GUIADO

### Objetivo:

Diseñar, construir, probar y documentar un módulo SOAP independiente que se integre con la aplicación monolítica de librería desarrollada previamente en Node.js + PostgreSQL, sin modificar el código del monolito. El módulo deberá utilizar Flask + psycopg2, publicar un contrato WSDL, construir manualmente los sobres SOAP en XML, consultar y registrar información en la base de datos PostgreSQL y ser consumido desde las aplicaciones de escritorio de clasificación.

El propósito del ejercicio es que el estudiante trabaje como ingeniero: analizar el sistema existente, definir el contrato antes de programar, proteger los datos del monolito, implementar una integración interoperable, probar casos normales y de error, justificar decisiones técnicas y producir evidencia reproducible del resultado.

- Sistema existente: monolito de librería Node.js + PostgreSQL.
- Nuevo componente: módulo SOAP independiente en Flask/Python.
- Contrato: WSDL + tipos XSD.
- Mensajería: SOAP Envelope XML construido manualmente con xml.etree.
- Persistencia: acceso directo y controlado a PostgreSQL mediante psycopg2.
- Cliente: aplicación de escritorio desarrollada en ejercicios anteriores.
- Publicación de evidencias: <https://ubiquitous.udem.edu/~iac-matricula/ejercicio04/>

**Importante:** el módulo SOAP es independiente del monolito, pero comparte la base de datos para este ejercicio académico. No debes modificar el código Node.js ni alterar las tablas existentes del monolito.

## Parte 1. Analizar el problema y definir el alcance

### 1. Revisar el sistema existente

Antes de escribir código, inspecciona la aplicación de librería y la base de datos. Identifica qué información necesita el nuevo módulo y qué información debe permanecer privada.

- Localiza y revisa las tablas books, concepts, book\_concepts y categories.
- Identifica claves primarias, claves foráneas, restricciones y columnas relevantes.
- Determina qué datos son de sólo lectura para el módulo SOAP y cuáles serán generados por el nuevo módulo.
- No reutilices la tabla users del monolito para los clasificadores del módulo SOAP.

### 2. Definir el scope del módulo SOAP

Documenta claramente qué está incluido y qué está fuera del alcance. Como mínimo, el módulo deberá:

- Consultar conceptos pendientes del catálogo real.
- Registrar clasificaciones Cloud: IaaS, PaaS, SaaS y FaaS.
- Consultar el progreso de clasificación de un usuario.
- Registrar clasificadores/clientes atendidos y contabilizar peticiones.
- Detectar clasificaciones duplicadas y responder mediante SOAP Fault.

- Mantener el monolito funcionando sin cambios.

Fuera del scope de esta etapa: migrar el monolito, crear una API REST, reemplazar PostgreSQL, modificar el esquema funcional existente o utilizar Spyne/Zeep para ocultar la construcción del Envelope en la implementación inicial.

## Parte 2. Diseñar la integración antes de programar

### 3. Diseñar la macro-arquitectura

Crea un diagrama que represente, como mínimo, el siguiente flujo lógico:

*Aplicación de escritorio → Cliente SOAP → HTTP POST/XML → Módulo SOAP Flask → psycopg2 → PostgreSQL*

En el mismo diagrama muestra que el monolito Node.js continúa accediendo a PostgreSQL de forma independiente. Identifica claramente las fronteras de cada aplicación, el protocolo de comunicación y qué componente es propietario de cada responsabilidad.

### 4. Registrar decisiones de ingeniería

Para cada decisión relevante utiliza el formato:

**Necesidad → Decisión → Justificación → Ventajas → Limitaciones**

Documenta al menos las decisiones sobre: Flask/Python, SOAP, WSDL/XSD, acceso a PostgreSQL, tablas propias, construcción manual del XML, autenticación, manejo de errores e interoperabilidad.

## Parte 3. Preparar el proyecto y el ambiente

### 5. Crear la estructura del módulo

Crea un proyecto independiente, por ejemplo: `library_soap_service`. Organiza el código para separar contrato, acceso a datos, lógica del servicio, seguridad y pruebas.

```
library_soap_service/
├── app.py
├── config/
├── db/
├── soap/
│   ├── service.py
│   ├── envelope.py
│   ├── faults.py
│   └── security.py
├── wsdl/
│   └── library-classifier.wsdl
├── sql/
│   └── soap_module.sql
├── tests/
├── .env.example
├── requirements.txt
└── README.md
```

## 6. Configurar el ambiente Python

Crea un ambiente virtual e instala únicamente las dependencias necesarias. La implementación inicial del SOAP deberá construir y leer XML manualmente; no uses frameworks que generen automáticamente los sobres.

```
python -m venv .venv
# Windows
.venv\Scripts\activate
# Linux
source .venv/bin/activate
pip install flask psycopg2-binary python-dotenv
```

Guarda credenciales y parámetros de conexión en variables de entorno. No publiques contraseñas, tokens ni archivos .env.

## Parte 4. Diseñar la persistencia del módulo SOAP

### 7. Crear tablas propias sin alterar las existentes

Diseña un script SQL para crear las tablas clasificadores, clasificaciones\_cloud y clientes\_servidos. Estas tablas pueden referenciar books.isbn y concepts.id cuando corresponda, pero no deben modificar la estructura de las tablas existentes.

- clasificadores: identidad del usuario del cliente SOAP y datos mínimos necesarios.
- clasificaciones\_cloud: concepto, libro, clasificador, modelo Cloud, fecha y datos de auditoría necesarios.
- clientes\_servidos: tipo de cliente de escritorio, identificador y número de peticiones atendidas.

Agrega PK, FK, UNIQUE, CHECK e índices que sean técnicamente justificables. Incluye una restricción que impida que el mismo clasificador registre dos veces el mismo concepto.

### 8. Aplicar mínimo privilegio

El usuario PostgreSQL utilizado por el módulo SOAP debe tener únicamente los permisos necesarios. Documenta qué tablas puede consultar y en cuáles puede insertar/actualizar. No uses el superusuario postgres desde la aplicación.

## Parte 5. Diseñar el contrato WSDL

### 9. Definir las operaciones obligatorias

Diseña primero el contrato y después la implementación. El WSDL deberá definir como mínimo:

- **ObtenerConceptosPendientes:** devuelve conceptos pendientes con la información necesaria del libro y categoría.
- **RegistrarClasificacion:** recibe clasificador, concepto y modelo de servicio; valida y registra la clasificación.
- **ObtenerProgresoUsuario:** devuelve totales clasificados y pendientes asociados al usuario.

Define mensajes de entrada/salida, tipos XSD, portType/interface, binding y endpoint. Utiliza tipos específicos y evita exponer columnas internas que el cliente no necesita.

## 10. Auditar el contrato

Antes de implementar, crea una tabla de auditoría del contrato:

| Dato                         | ¿Se expone?    | Justificación                                                     |
|------------------------------|----------------|-------------------------------------------------------------------|
| ISBN / título / concepto     | Sí             | Necesarios para que el cliente identifique qué está clasificando. |
| Credenciales de BD           | No             | Información sensible interna.                                     |
| Campos internos de auditoría | Sólo si aplica | Exponer únicamente lo necesario para el contrato.                 |

**La regla de ingeniería es:** el contrato expone capacidades y datos necesarios, no la estructura completa de la base de datos.

## Parte 6. Implementar SOAP manualmente

### 11. Construir y procesar el SOAP Envelope

Implementa el endpoint SOAP en Flask. Recibe HTTP POST, analiza el XML, identifica la operación solicitada, valida datos, ejecuta la lógica y construye la respuesta SOAP.

- Utiliza `xml.etree.ElementTree` para leer y construir XML.
- Diferencia Envelope, Header y Body.
- Maneja namespaces correctamente.
- No concatenes XML sin escapar valores.
- Valida campos obligatorios y modelos permitidos.

Ejemplo conceptual de Envelope:

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Header>...</soap:Header>
  <soap:Body>
    <RegistrarClasificacion>...</RegistrarClasificacion>
  </soap:Body>
</soap:Envelope>
```

### 12. Implementar acceso a PostgreSQL

Crea una capa de acceso a datos independiente. Utiliza consultas parametrizadas (Stored Procedures y Vistas), transacciones cuando una operación modifique varias tablas y rollback ante errores. No construyas SQL concatenando entradas del usuario.

ObtenerConceptosPendientes deberá integrar información real de `book_concepts` + `concepts` + `books` + `categories`, respetando el esquema existente.

## Parte 7. Implementar errores mediante SOAP Fault

### 13. Diseñar casos de error

El servicio no deberá devolver únicamente texto genérico cuando algo falla. Implementa SOAP Fault con información suficiente para que el cliente pueda distinguir errores de entrada, conflictos y errores internos.

| Caso                                   | Comportamiento esperado                                        |
|----------------------------------------|----------------------------------------------------------------|
| Concepto inexistente                   | SOAP Fault de cliente con mensaje claro.                       |
| Modelo distinto de IaaS/PaaS/SaaS/FaaS | SOAP Fault de validación.                                      |
| Clasificación duplicada                | SOAP Fault asociado a conflicto 409.                           |
| XML inválido                           | SOAP Fault de cliente; no ejecutar SQL.                        |
| Falla de PostgreSQL                    | SOAP Fault de servidor; registrar detalle técnico sólo en log. |

No expongas stack traces, contraseñas, rutas internas o consultas SQL en el Fault enviado al cliente.

## Parte 8. Integrar las aplicaciones de escritorio

### 14. Agregar modo cliente SOAP

Modifica las aplicaciones de escritorio desarrolladas previamente para incorporar un modo cliente SOAP. La GUI deberá capturar nombre, apellidos y correo, además del texto o concepto a clasificar.

- Construir el Envelope de solicitud.
- Enviar la petición al endpoint SOAP.
- Procesar la respuesta y mostrar la clasificación en la GUI.
- Procesar SOAP Fault y mostrar un mensaje comprensible al usuario.
- Evitar que la GUI conozca directamente detalles de PostgreSQL.

La aplicación de escritorio deberá comunicarse con el módulo SOAP; no deberá conectarse a PostgreSQL para ejecutar la clasificación.

## Parte 9. Probar el sistema como ingeniero

### 15. Crear un plan de pruebas

Ejecuta pruebas positivas y negativas. Cada prueba debe tener entrada, resultado esperado, resultado obtenido, evidencia y conclusión.

| ID  | Prueba                        | Resultado esperado | Resultado obtenido | Estado |
|-----|-------------------------------|--------------------|--------------------|--------|
| P01 | Obtener conceptos pendientes  | Lista válida       |                    |        |
| P02 | Registrar IaaS/PaaS/SaaS/FaaS | Registro exitoso   |                    |        |
| N01 | Repetir clasificación         | SOAP Fault 409     |                    |        |
| N02 | Concepto inexistente          | SOAP Fault         |                    |        |
| N03 | Modelo inválido               | SOAP Fault         |                    |        |

## 16. Verificar persistencia e integración

Clasifica conceptos reales de varios libros y categorías. Verifica directamente en PostgreSQL que los registros llegaron a clasificaciones\_cloud y que clientes\_servidos refleja el tipo de cliente y el número de peticiones atendidas.

Conserva evidencia del WSDL, solicitudes/respuestas XML, GUI, SOAP Fault y consultas de verificación en PostgreSQL.

## Parte 10. Documentar la construcción del módulo SOAP

### 17. Elaborar documentación técnica

Documenta y justifica las principales decisiones de ingeniería utilizadas en el desarrollo del módulo. La documentación deberá explicar, como mínimo:

- Arquitectura seleccionada y justificación.
- Diseño: macro-arquitectura, patrón de diseño y organización del código.
- Organización de módulos y componentes.
- Contrato WSDL, tipos XSD y operaciones.
- Funcionalidades implementadas.
- Flujo de información entre usuario, aplicación de escritorio, módulo SOAP y base de datos.
- Autenticación, autorización y seguridad.
- Validación de datos y manejo de SOAP Fault.
- Persistencia, transacciones y permisos de PostgreSQL.
- Despliegue y configuración del servicio.
- Consideraciones de rendimiento, mantenimiento, interoperabilidad y escalabilidad.

Incluye diagramas que muestren: Web Monolito, módulo SOAP, aplicaciones de escritorio, contrato de comunicación y PostgreSQL.

**Para cada decisión relevante utiliza: Necesidad → Decisión → Justificación → Ventajas → Limitaciones.**

## TRABAJO EN CASA

Las siguientes actividades se realizarán individualmente fuera del laboratorio. Todas las evidencias deberán integrarse en la página web del ejercicio en [ubiquitous.udem.edu](http://ubiquitous.udem.edu).

## Tarea 1. Estadísticas por modelo + WS-Security

### Objetivo

Extender el contrato sin romper las operaciones existentes y aplicar autenticación a una operación sensible.

### Actividad

- Agrega ObtenerEstadisticasPorModelo para obtener el conteo por IaaS/PaaS/SaaS/FaaS.

- Actualiza el WSDL y XSD conservando compatibilidad con las operaciones anteriores.
- Protege la operación con un mecanismo WS-Security basado en usuario/contraseña apropiado para el ejercicio.
- No almacenes contraseñas en texto plano ni publiques credenciales en las evidencias.

### Evidencias

- WSDL actualizado.
- Solicitud autenticada y respuesta.
- Prueba con credenciales incorrectas.
- Capturas y explicación de la decisión de seguridad.

## Tarea 2. SOAP Fault y experiencia de usuario

### Objetivo

Diseñar el manejo de errores de forma consistente entre servicio y cliente.

### Actividad

- Provoca intencionalmente una clasificación duplicada.
- Verifica el SOAP Fault asociado al conflicto 409.
- Modifica la GUI para interpretar el Fault y mostrar un mensaje útil sin exponer detalles técnicos.
- Prueba además concepto inexistente, XML inválido y modelo Cloud inválido.

### Evidencias

- Envelope de error.
- Captura de la GUI.
- Tabla de pruebas negativas.
- Explicación de qué información se muestra al usuario y qué información queda sólo en logs.

## Tarea 3. Auditoría del contrato WSDL

### Objetivo

Evaluar el contrato como una superficie pública del sistema y justificar qué información se expone.

### Actividad

Analiza operación por operación y documenta qué columnas del monolito se exponen, cuáles se transforman y cuáles se ocultan deliberadamente. Identifica posibles riesgos de seguridad o acoplamiento.

### Evidencias

- Tabla dato → operación → exposición → justificación.
- Diagrama del contrato.
- Al menos tres riesgos identificados y su mitigación.
- Conclusión sobre contrato estricto y mantenibilidad.



## Tarea 4. Interoperabilidad

### Objetivo

Demostrar que el WSDL permite que un cliente en otro lenguaje consuma el mismo servicio sin conocer su implementación interna.

### Actividad

Genera un cliente a partir del WSDL usando una tecnología diferente a la del servidor, por ejemplo wsimport (Java), dotnet-svcutil (.NET) o zeep (Python). Consume al menos ObtenerConceptosPendientes y RegistrarClasificacion.

### Evidencias

- Herramienta y lenguaje utilizados.
- Proceso de generación del cliente.
- Código relevante.
- Capturas de ejecución.
- Comparación entre cliente manual y cliente generado desde WSDL.

## Tarea 5. Medición técnica y reflexión SOAP

### Objetivo

Obtener métricas que puedan utilizarse en la siguiente sesión para comparar SOAP con REST.

### Actividad

- Registra el tiempo aproximado de desarrollo del módulo SOAP.
- Cuenta líneas de código relevantes del servicio y del cliente.
- Mide en bytes al menos una solicitud y respuesta de RegistrarClasificacion.
- Registra qué porcentaje aproximado corresponde a estructura XML frente a datos de negocio.
- Conserva estas métricas para compararlas posteriormente con REST.

### Preguntas para reflexionar

- ¿Qué pasaría si agregas un campo obligatorio a RegistrarClasificacion sin actualizar a los clientes?
- ¿Qué información del WSDL podría ser útil para un atacante y qué no debería exponerse?
- ¿Por qué el tipado XSD aporta certeza al contrato?
- ¿Qué partes del Envelope fueron boilerplate y cuáles dependieron de la operación?
- ¿Cuándo justificarías el overhead de XML frente a un formato más ligero?
- ¿Por qué un SOAP Fault es parte del contrato y no sólo un error HTTP?
- ¿Cómo debe reaccionar una GUI ante un Fault de cliente frente a un Fault de servidor?
- ¿Qué riesgo existe porque el módulo SOAP y el monolito compartan la misma base de datos?
- ¿Qué ocurriría si el monolito renombra books.isbn? ¿Cómo reducirías ese acoplamiento?
- ¿Dónde debe aplicarse autenticación y qué problema resuelve WS-Security?
- ¿Es suficiente confiar en el correo enviado por el cliente para identificar al usuario?
- ¿En qué escenarios empresariales seguirías eligiendo SOAP en 2026?

## Parte 11. Reporte técnico en página web

### 18. Publicar el ejercicio en ubiquitous.udem.edu

Crea una página web que funcione como reporte técnico navegable de todo el ejercicio. La URL deberá seguir la estructura:

**`https://ubiquitous.udem.edu/~iac-matricula/ejercicio03/`**

La página no es sólo una galería de capturas. Debe permitir que otro ingeniero comprenda el problema, reproduzca el trabajo y evalúe las decisiones tomadas.

### 19. Estructura mínima del reporte web

- 1. Objetivo y scope del ejercicio.
- 2. Arquitectura inicial y arquitectura final.
- 3. Descripción del monolito y del nuevo módulo SOAP.
- 4. Diseño de datos y tablas propias del módulo.
- 5. Contrato WSDL y tipos XSD.
- 6. Operaciones implementadas.
- 7. Ejemplos de SOAP Request, SOAP Response y SOAP Fault.
- 8. Seguridad y WS-Security.
- 9. Aplicaciones de escritorio como clientes SOAP.
- 10. Interoperabilidad con un segundo lenguaje.
- 11. Plan de pruebas y resultados.
- 12. Evidencias de PostgreSQL.
- 13. Decisiones de ingeniería y trade-offs.
- 14. Problemas encontrados y soluciones.
- 15. Métricas para comparación futura con REST.
- 16. Uso de IA y prompts utilizados.
- 17. Conclusiones.
- 18. Descargas: WSDL, XSD si aplica, SQL, código fuente y README.

### 20. Estructura sugerida en el servidor

```
~/html/ejercicio04/  
├─ index.html  
├─ css/  
├─ img/  
├─ evidencias/  
├─ wsdl/  
├─ sql/  
├─ docs/  
└─ descargas/
```

Verifica desde una ventana privada que la página, imágenes, CSS, WSDL, documentos y descargas sean accesibles. No publiques .env, contraseñas, tokens, claves SSH, cadenas de conexión ni datos sensibles.

## Entrega del trabajo en casa

La entrega se considerará completa cuando la página del ejercicio esté publicada y contenga una sección claramente identificable para cada actividad:

- Tarea 1 → Estadísticas por modelo + WS-Security.
- Tarea 2 → SOAP Fault y experiencia de usuario.
- Tarea 3 → Auditoría del contrato WSDL.
- Tarea 4 → Interoperabilidad.
- Tarea 5 → Medición técnica y reflexión SOAP.
- Reporte técnico → documentación completa del ejercicio.

Además, entrega el código fuente organizado, README.md, requirements.txt, WSDL/XSD, script SQL del módulo y evidencias suficientes para reproducir las pruebas.

## Consideración sobre el uso de Inteligencia Artificial

Se permite y fomenta el uso de herramientas de IA como apoyo para investigar, revisar código, detectar errores, proponer pruebas y mejorar documentación; sin embargo, su utilización deberá documentarse. Registra los prompts relevantes utilizados y explica qué aceptaste, qué modificaste y qué rechazaste.

El estudiante continúa siendo responsable de:

**comprender → revisar → probar → validar → corregir → justificar**

No se evaluará quién genera más código con IA, sino quién demuestra mayor capacidad para utilizarla como herramienta de ingeniería, verificar sus resultados y defender técnicamente sus decisiones.

## Criterios de finalización

- El monolito Node.js continúa funcionando sin modificaciones.
- El servicio SOAP publica un WSDL válido y ejecuta las operaciones requeridas.
- Las aplicaciones de escritorio consumen el servicio SOAP.
- Los datos se registran correctamente en PostgreSQL.
- Los errores relevantes producen SOAP Fault controlados.
- La seguridad y los permisos están documentados y probados.
- Existe evidencia de interoperabilidad.
- La página web del ejercicio está publicada y accesible.
- Todas las decisiones importantes están justificadas desde una perspectiva de ingeniería.
- Conclusiones personales del ejercicio.